BAB 5
CONTROL FLOW

5.1. Capaian Pembelajaran

- Mahasiswa mampu menjelaskan tentang konsep control flow pada pemrograman dart.
- Mahasiswa mampu menerapkan *if and else* pada pemrograman dart.
- Mahasiswa mampu menerapkan *for loops* pemrograman dart.
- Mahasiswa mampu menerapkan *while* pada pemrograman dart.
- Mahasiswa mampu menerapkan *do-while* pada pemrograman dart.
- Mahasiswa mampu menerapkan *break and continue* pada pemrograman dart.
- Mahasiswa mampu menerapkan *switch and case* pada pemrograman dart.

5.2. Pendahuluan

Control flow merupakan landasan fundamental dalam pemrograman yang memungkinkan pengembang untuk mengatur alur eksekusi program. Dalam bahasa pemrograman Dart, kontrol alur dari suatu program dapat dicapai melalui sejumlah struktur seperti penggunaan kondisional, perulangan, dan pengambilan keputusan. Pemahaman yang mendalam tentang control flow sangat penting bagi pengembang untuk mengendalikan logika program, membuat keputusan berdasarkan kondisi tertentu, serta mengeksekusi perintah-perintah secara berulang. Dalam bab ini akan membahas lebih dalam mengenai struktur control flow dalam bahasa Dart dan bagaimana penggunaannya memengaruhi alur dari suatu program, beberapa control flow yang akan dipelajari adalah konsep dasar tentang *if else, for, while, do-while, break and continue* dan *switch case*. Dalam Bahasa pemrograman lain,struktur penulisan dari control flow tidak berbeda jauh, bahkan secara konsep dan penulisan kerangka dasar kode adalah sama, hanya saja perlu menyesuaikan jika penggunaannya berkolaborasi dengan bagian lain seperti penggunaan variabel dalam control flow, penulisan fungsi dan lain sebagainya.

5.3. If and Else

If adalah salah satu bentuk struktur kontrol yang memungkinkan eksekusi kode berdasarkan kondisi yang diberikan. Ini digunakan untuk membuat keputusan atau percabangan dalam program. Struktur dasar dari if dalam Dart adalah sebagai berikut:

```
void main() {
  var libur = true;

  if (libur == true) {
    print("Hari ini Libur");
}
```

Dari kode diatas, print diletakkan diantara fungsi if, yang bermakna bahwa print akan menampilkan teks ”Hari ini Libur” jika variabel libur bernilai benar (***true***). Namun print ini tidak akan berfungsi jika variabel libur bernilai salah (***false***), artinya tidak aka nada teks yang ditampilkan. Kondisi if tunggal semacam ini dapat kita gunakan untuk kondisi dimana kita tidak membutuhkan adanya jawaban yang salah (karena hanya cukup dengan tidak perlu ditampilkan), namun jika kita memerlukan jawaban salah untuk ditampilkan maka wajib memberikan pasangan terhadap if.

If pada umumnya dipasangkan dengan else, biasanya ini diperlukan jika memang terdapat opsi jawaban yang salah untuk ditampilkan. Perhatikan contoh berikut:

```
void main() {
  var libur = true;

  if (libur == true) {
    print("Hari ini Libur");
  } else {
    print("Hari ini Tidak Libur");
  }
}
```

Dari contoh diatas dapat disimpulkan bahwa ketika jawaban benar ataupun jawaban salah, akan tetap ditampilkan ke layer, meskipun dengan output yang berbeda. Fungsi if akan melakukan validasi terhadap nilai pada variabel libur, jika memang benar bahwa variabel libur bernilai true, maka pernyataan “Hari ini Libur” yang akan dijalankan, artinya validasi tersebut tepat. Berbeda jika kita ubah nilai variabel libur menjadi false, maka bagian else yang akan dijalankan (atau kondisi yang tidak sesuai dengan kondisi sebenarnya).

Kita juga dapat mengembangkan ini menjadi lebih dari dua kondisi, seperti contoh pada kode berikut:

```
void main() {
  var nilai = 30;

  if (nilai >= 90) {
    print("A");
  } else if (nilai < 90) {
    print("B");
  } else if (nilai < 80) {
    print("C");
  } else if (nilai < 60) {
    print("D");
  }
}
```

Kode diatas jika dijalankan, maka tidak akan terjadi masalah berupa *error* terhadap program, Namun kode tersebut masih terdapat kemungkinan permasalahan terhadap fungsionalitasnya, Misalkan jika kita mengubah nilai menjadi lebih dari 100 atau nilai negatif, maka kode tersebut belum dapat mengidentifikasinya, kasus seperti diatas dapat diatasi apabila dikombinasikan dengan operator perbandingan yang telah kita pelajari pada tabel 4 dan operator logika pada tabel 5..

Perhatikan modifikasi kode program berikut:

```
void main() {
  var nilai = -101;

  if (nilai >= 90 && nilai <= 100) {
    print("A");
  } else if (nilai >= 80 && nilai < 90) {
    print("B");
  } else if (nilai >= 60 && nilai < 80) {
    print("C");
  } else if (nilai >= 0 && nilai < 60) {
    print("D");
  } else {
    print("nilai $nilai tidak diakui");
  }
}
```

Dengan menambahkan operator perbandingan dan operator logika seperti diatas, tentu akan lebih baik dalam mengidentifikasi nilai yang diinputkan karena telah ditetapkan batasan-batasan nilai yang diperbolehkan untuk diolah oleh program.

Yang menarik dari Bahasa Dart yaitu telah mendukung penulisan *conditional expression* dalam penulisan satu baris kode. Seperti contoh Berikut:

```
void main() {
  var nilaiMin = 70;
  var nilaiKu = 65;

  var ket = nilaiKu > nilaiMin ? "Lulus" : "Tidak Lulus";
  print(ket);
}
```

Dengan model penulisan seperti diatas akan meminimalisir penulisan baris kode program.

If juga dapat digunakan didalam if, atau yang umum disebut if bersarang (nested if). Biasanya ini diperlukan untuk melakukan pengecekan kondisi tambahan. Penggunaannya seperti berikut:

```
if (jk == pria) {
  if (umur <= 18) {
    belum dewasa
  }
}
```

Kode diatas dapat dikembangkan dengan berbagai if serta digabungkan dengan penggunaan else.

Latihan

Buatlah aplikasi Flutter yang menampilkan kategori umur berdasarkan nilai variabel usia. Gunakan pernyataan if-else untuk mengontrol kategori umur sesuai dengan aturan berikut:

Jika usia kurang dari 12, tampilkan teks "Anak-anak".

Jika usia antara 12 dan 17 (inklusif), tampilkan teks "Remaja".

Jika usia antara 18 dan 64 (inklusif), tampilkan teks "Dewasa".

Jika usia 65 atau lebih, tampilkan teks "Lansia".

5.4. For

For adalah salah satu dari beberapa bentuk loop yang memungkinkan untuk melakukan iterasi atau perulangan sejumlah tertentu. Ada beberapa variasi dari loop for yang dapat digunakan, seperti for-in dan forEach, yang memungkinkan iterasi melalui berbagai jenis struktur data.

Rumus dasar dari for dalam dart adalah sebagai berikut:

```
for (var i = 0; i < limit; i++) {
  // kode yang akan diulang sejumlah limit kali
}
```

Ketika dalam suatu program membutuhkan penulisan angka berurutan dari 1 sampai 5 dengan manual, maka yang akan dilakukan adalah menulis dengan cara:

```
void main() {
  print("1");
  print("2");
  print("3");
  print("4");
  print("5");
}
```

Kebutuhan menampilkan 5 angka tersebut masih terlihat sederhana, jika angka yang harus ditampilkan di kisaran ratusan atau bahkan ribuan, kita memerlukan For ini untuk mengantisipasi penulisan baris kode hingga ratusan atau ribuan tersebut.

Penggunaan for adalah seperti contoh berikut:

```
void main() {
  for (int angka = 1; angka <= 5; angka++) {
    print(angka);
  }
}
```

Output:

```
1
2
3
4
5
```

Jika kita membutuhkan angka 1 sampai dengan 1000, kita hanya mengubah nilai 5 menjadi 1000. Atau jika yang ingin ditampilkan adalah nilai tertentu selain angka, dapat kita modifikasi menjadi seperti berikut:

```
void main() {
  var bahasa = "dart";
  for (int angka = 1; angka <= 5; angka++) {
    print(bahasa);
  }
}
```

Kode diatas akan menampilkan ”dart” sebanyak lima kali.

Latihan

Buatlah program Flutter yang menggunakan for loop untuk menampilkan angka 1 hingga 50 dalam bentuk baris pada layar.

5.5. For-in

For juga dapat digunakan untuk mengulangi item dalam koleksi seperti list, set, atau map, penggunaannya seperti berikut:

```
var myList = [1, 2, 3, 4, 5];
for (var item in myList) {
  // kode yang dijalankan untuk setiap item dalam myList
}
```

```
var mySet = {1, 2, 3, 4, 5};
for (var item in mySet) {
  // kode yang dijalankan untuk setiap item dalam mySet
}
```

```
var myMap = {'a': 1, 'b': 2, 'c': 3};
for (var key in myMap.keys) {
  var value = myMap[key];
  // kode yang dijalankan untuk setiap pasangan kunci-nilai dalam myMap
}
```

5.6. ForEach

Loop forEach memungkinkan untuk mengulangi setiap elemen dalam koleksi dengan menggunakan fungsi callback dalam koleksi seperti list, set, atau map.

```
var myList = [1, 2, 3, 4, 5];
myList.forEach((item) {
  // kode yang dijalankan untuk setiap item dalam myList
});
```

```
var mySet = {1, 2, 3, 4, 5};
mySet.forEach((item) {
  // kode yang dijalankan untuk setiap item dalam mySet
});
```

```
var myMap = {'a': 1, 'b': 2, 'c': 3};
myMap.forEach((key, value) {
  // kode yang dijalankan untuk setiap pasangan kunci-nilai dalam myMap
});
```

5.7. While

Selain menggunakan For, Dart juga mendukung penggunaan While, fungsinya sama yaitu menjalankan perulangan dari kode yang terdapat didalam while ketika suatu kondisi bernilai benar atau true. Rumus penggunaannya seperti berikut:

```
while (condition) {
  // kode yang akan diulang selama kondisi benar
}
```

Perhatikan contoh berikut:

```
void main() {
  var angka = 1;
  while (angka <= 5) {
    print(angka);
    angka++;
  }
}
```

Ini terjadi karena while akan terus berjalan selama nilai angka kurang dari atau sama dengan 5. Ketika nilai angka adalah 1, maka pernyataan print(angka) akan mencetak nilai 1, kemudian nilai angka akan ditambah 1 sehingga nilainya menjadi 2. Proses ini akan terus berlanjut sampai angka mencapai 6, saat itulah kondisi angka <= 5 menjadi salah dan loop berhenti.

Untuk memastikan bahwa kondisi dalam while pada suatu titik akan berubah sehingga loop bisa berakhir. Jika tidak, Anda bisa mendapatkan apa yang disebut sebagai "*infinite loop*" yang akan terus berjalan tanpa berhenti.

Satu hal penting untuk diperhatikan adalah memastikan bahwa kondisi awal loop while sudah benar saat pertama kali dievaluasi, jika tidak, loop mungkin tidak pernah berjalan. Jadi pastikan kondisinya sesuai dengan logika yang Anda inginkan.

Latihan

Implementasikan program Flutter yang menggunakan while loop untuk mencetak kelipatan 3 dari 3 hingga 15.

5.8. Do-While

Merupakan struktur pengulangan pada Dart yang mirip dengan while, tetapi dengan perbedaan bahwa blok kode dieksekusi terlebih dahulu sebelum pengecekan kondisi. Ini berarti bahwa setidaknya satu iterasi akan dijalankan sebelum kondisi dicek, dan selama kondisi tetap benar, loop akan terus berjalan. Ini membedakannya dari loop while biasa, di mana pengecekan kondisi dilakukan sebelum eksekusi blok kode.

Tahapan alur do-while pertama kali adalah blok kode dijalankan tanpa mempertimbangkan kondisi, setelah blok kode dieksekusi, barulah kondisi dievaluasi. Jika kondisi bernilai true, maka blok kode akan terus dieksekusi, namun jika kondisi bernilai false maka eksekusi akan keluar dari loop.

Berikut syntax dasarnya:

```
do {
  // kode yang akan dijalankan setidaknya sekali
} while (condition);
```

Untuk contoh implementasinya dapat dilihat pada kode berikut:

```
void main() {
  var count = 1;

  do {
    print('Nilai count: $count');
    count++;
  } while (count <= 5);
}
```

Hasil dari contoh di atas akan mencetak nilai dari 1 hingga 5. Loop do-while akan menjalankan blok kode terlebih dahulu, kemudian mengecek kondisi count <= 5. Jika kondisi tersebut benar, maka loop akan terus berjalan, jika tidak, maka loop berhenti.

Penting untuk dicatat bahwa dengan do-while, blok kode akan dijalankan setidaknya satu kali meskipun kondisi awalnya salah, karena pengecekan kondisi dilakukan setelah eksekusi blok kode pertama kali. Hal ini berbeda dengan while biasa yang dapat langsung meloncati eksekusi jika kondisi awalnya salah.

5.9. Switch and Case

Switch-case adalah struktur kontrol di Dart yang memungkinkan eksekusi kode yang berbeda berdasarkan nilai dari suatu variabel atau ekspresi tertentu. Ini digunakan ketika Anda memiliki beberapa kemungkinan nilai untuk suatu variabel dan ingin mengeksekusi blok kode yang berbeda berdasarkan nilai tersebut. Bagian ini mendukung fungsi yang mirip dengan penggunaan if, hanya saja switch case lebih unggul karena dapat melakukan cek pada banyak kondisi dengan penulisan yang lebih ringkas, dibandingkan harus menulis satu baris if untuk setiap satu kondisi. Berikut adalah struktur dasar dari switch-case di Dart:

```
switch (expression) {
  case value1:
    // kode yang dieksekusi jika expression sama dengan value1
    break;
  case value2:
    // kode yang dieksekusi jika expression sama dengan value2
    break;
  // case-case lainnya
  default:
    // kode yang dieksekusi jika expression tidak sama dengan nilai semua case
}
```

- **Expression**, nilai atau ekspresi yang akan dievaluasi.
- **case value1,** pernyataan yang menentukan kode yang akan dieksekusi jika expression sama dengan value1**.**
- **Break,** digunakan untuk keluar dari switch-case setelah sebuah case dieksekusi.
- **Default,** bagian opsional yang dieksekusi jika nilai expression tidak sama dengan nilai dari semua case.

```
void main() {
  var nilai = 'B';

  switch (nilai) {
    case 'A':
      print('Sangat Baik');
      break;
    case 'B':
      print('Baik');
      break;
    case 'C':
      print('Cukup');
      break;
    default:
      print('Nilai tidak valid');
  }
}
```

Dalam contoh ini, terdapat variabel nilai dengan nilai 'B'. Switch-case mengevaluasi nilai dari nilai dan mencocokkannya dengan case-case yang ada. Karena nilai adalah 'B', maka blok kode di bawah case 'B': yang dieksekusi, yaitu print('Baik');. Kemudian, karena ada break, eksekusi keluar dari switch-case. Jika nilai tidak cocok dengan nilai dari semua case yang ada, maka blok kode di dalam default akan dieksekusi.

Switch-case sering digunakan ketika terdapat beberapa kondisi atau nilai yang harus diuji, dan memungkinkan kode Anda untuk menjadi lebih terstruktur daripada penggunaan serangkaian statement if-else yang panjang.

Contoh lainnya:

```
void main() {
  final angka1 = 2;
  final angka2 = 5;
  final operator = "tambah";

  switch (operator) {
    case 'tambah':
      print(angka1 + angka2);
      break;
    case 'kurang':
      print(angka1 - angka2);
      break;
default:
      print('tidak ada pilihan tersebut');
  }
}
```

Latihan

Buatlah aplikasi Flutter sederhana yang meminta pengguna memasukkan nama bulan (dalam bentuk angka 1-12) dan kemudian menggunakan switch statement untuk menampilkan jumlah hari dalam bulan tersebut.